KG 구축 how 보완
단계 1~5: 자료에서 지식그래프까지

벤치마킹한 문헌의 원리와 우리 구현 과정, 그리고 다음 주 계획

2026.10.30 · 문경민

1. 이번에 바뀐 점

핵심

- 10/19 지적대로 단계마다 "문헌의 어떤 원리를 따라, 우리 자료로 어떻게 했고, 무엇이 만들어졌는지"를 실제 자료의 행으로 보였다. 단계 1~5 를 마치면 KG(Entity 264, 속성 616, 관계 418)가 완성된다(2 장, 6.3).
- 문헌 표기를 바로잡았다. OntoDSMS 는 Zhou 외(2023)가 만든 온톨로지의 이름이고 D2RQ 는 그 논문이 사용한 기존 도구이다. 전처리는 DDKG 의 기술을 쓰지 않았으므로 문헌 근거 없는 자료 준비로 고쳤다.
- 문헌의 원리 설명, 용어, CQ, 온톨로지 설계, 저장소 구조처럼 이미 정해진 내용은 부록으로 옮겼다.
- 검증(단계 6), 저장(단계 7), 검색·답변은 다음 주에 다룬다(8 장).

표 1. 10/16 자료와 비교

항목	10/16 자료	이번
단계 1·2	근거: DDKG 3.2.1, Zhang(2025). 처리와 결과를 표로	소스 구분은 DDKG 의 지식 출처 구분, 전처리는 문헌 근거 없음. 실제 행으로 지우는 행, 바뀌는 시각, 꺼내는 부분을 보임
단계 3	"OntoDSMS(Zhou 2023)는 … 연결한다". 표 → Class, 열 → 속성, 행 → 개체(원칙만), Class 별 결과 예	논문 · 온톨로지(OntoDSMS) · 도구(D2RQ) 구분. 원리 4 개, 승인 504 행 하나를 따라가는 과정 5 개, 결과물 CSV 3 개
단계 4	근거: DDKG 3.2.2~3.2.3, GraphAide. 운영행위 시각은 총방류량 변화로 정함	DDKG 3.2.2(규칙 기반 추출)만. 시각은 비고에 적힌 시각, 없으면 방류시작시간(측정값 보정 없앰). 실제 비고 5 건의 처리
단계 5	관계별 연결 방법 표. 변경 승인 → 원 승인의 운영행위 추정 연결(12 건)	기준 목록(댐 코드)에 대응시키는 원리와 과정, 연결 결과 418 개. 추정 연결은 없앰
결과	숫자 위주	단계별 결과 파일과 실제 줄, 최종 KG 그림(충주댐)
10/19 질문	—	DDKG 가 무엇인가 → 부록 E.1 · 전처리 후 무엇이 남나 → 3.2 · 매핑이 어떻게 작동하나 → 4 장 · 댐 기본 자료 → 4.3 · AP 와 APR → 과정 4, 7 장 · 연결이 실제로 되나 → 6 장

2. KG 구축 전체 흐름

- 자료 5 가지에서 출발해 다섯 단계를 거치면 KG 가 만들어진다. 각 단계의 결과물은 CSV 파일이며, 다음 단계의 입력이 된다.

자료 5 가지 → 단계 1·2 자료 준비 → 단계 3 구조화 자료 매핑 → 단계 4 비정형 텍스트 추출 → 단계 5 개체 해소·연결 → KG Entity 264 관계 418

표 2. 단계별 입력, 결과물, 따른 원리

단계	입력	결과물	따른 원리
1·2 자료 준비	승인 기록 CSV, K-water 일자료·시간자료, 규정 XML	자료 → Class 대응, 정리된 승인 80 행, 측정 저장소, 별표 3 원문	소스 구분: DDKG 의 지식 출처 구분 전처리: 문헌 근거 없음
3 구조화 자료 매핑	승인 80 행, K-water 일자료·시간자료	Approval 80, EvidenceSource 84, Dam 4, HMS 24 와 속성	D2RQ 의 매핑 원리 (Zhou 외 2023 이 사용)
4 비정형 텍스트 추출	승인의 비고, 별표 3 원문	Operation 68, Criterion 3 과 속성	DDKG 의 규칙 기반 추출
5 개체 해소·연결	단계 3·4 의 Entity	관계 418	GraphAide 의 개체 중의성 해소

각 원리의 자세한 설명은 부록 E 에 있다.

3. 자료 준비 (단계 1 소스 구분 · 단계 2 전처리)

3.1 소스 구분: 자료마다 처리 방법과 채우는 Class 를 정한다

- 따른 원리: DDKG(Huang 외 2026)는 플랫폼의 정형 표로 c1~c4(Dam 등)를, 보고서의 비정형 텍스트로 c5~c7(위험 형태 등)을 채운다. Class 마다 출처를 정하고, 정형 표인지 비정형 텍스트인지에 따라 처리를 나눈다(부록 E.1).
- 우리 자료 5 가지도 같은 원리로 나누었다(그림 1). 승인 기록 CSV 와 K-water 일자료·시간자료는 열이 정해진 정형 표라서 열 값을 그대로 옮긴다(단계 3).
- 승인 기록의 비고 열은 표 안에 있지만 "(실제방류는 20 시부터)" 같은 문장이고, 규정 별표 3 은 선 문자(|, –)로 그린 글자라서 칸을 그대로 읽을 수 없다. 둘 다 글자를 읽어 값을 뽑아야 하므로 비정형으로 나누었다(단계 4).
- 그래서 비고 열은 두 번 쓰인다. 단계 3 에서는 승인 표의 한 칸으로서 원문을 그대로 옮기고, 단계 4 에서는 그 문장을 읽어 운영행위(Operation)를 뽑는다.



그림 1. 소스 구분 — 자료 5 가지의 형태, 처리 방법, 채우는 Class

3.2 전처리: 원본은 그대로 두고, 쓸 수 있는 형태로 정리한다

- DDKG 의 전처리는 형식이 제각각인 보고서를 마크다운으로 바꾸고(이미지는 OCR) 정규식으로 필요한 구간만 잘라내는 것이다. 우리 자료는 정형 표이거나 이미 글자(XML)라서 이 기술을 쓰지 않았다. 따라서 전처리는 문헌을 따른 방법이 아니라 자료 준비이다.
- 원칙은 원본 값은 바꾸지 않고, 지우거나 바꾼 것은 기록으로 남기는 것이다.

승인 기록: 중복 행을 지우고 대상 댐의 행을 고른다

- 원본은 한강수계 16 개 시설의 승인 3,929 행이다. 행마다 원본 행 번호와 출처 열(기관, 파일명 등)을 붙인다. 원본 행 번호는 단계 3 에서 Entity 의 출처가 원본의 어느 행인지 가리키는 데 쓰인다.

- 그림 2 에서 2220 행은 2218 행과 값이 모두 같다. 같은 승인이 두 번 입력된 것으로 보고 2220 을 지우고, (지운 번호, 남긴 번호, 이유)를 로그에 남긴다. 2219 는 방류시작시간이 달라 남긴다. 이렇게 지운 행은 전체 7 행이다.
- 괴산댐, 팔당댐처럼 대상 4 개 댐(충주, 소양강, 횡성, 광동)이 아닌 행은 쓰지 않는다. 남은 80 행이 단계 3 의 입력이다.

승인 기록 원본 — 한강수계 16개 시설, 3,929행 중 순차번호 2216~2222

원본 행 번호 (추가)	순차번호	관측소명	승인일	방류시작시간	접수 방류량	비고	출처 (추가)	처리
2216	2216	괴산댐	2016-07-04	2016-07-04 15:30	736	(감소방류 -295㎥/s)	…	대상 외
2217	2217	팔당댐	2016-07-04	2016-07-04 19:00	899	(초기수문방류 130㎥/s)	…	대상 외
2218	2218	광동	2016-07-04	2016-07-04 18:00	200		…	남김
2219	2219	광동	2016-07-04	2016-07-04 18:40	200		…	남김
~~2220~~	~~2220~~	~~광동~~	~~2016-07-04~~	~~2016-07-04 18:00~~	~~200~~		~~…~~	지움 = 2218
2221	2221	청평댐	2016-07-04	2016-07-04 21:30	755	(초기수문방류 99㎥/s)	…	대상 외
2222	2222	팔당댐	2016-07-04	2016-07-04 19:20	1757	(증가방류 858㎥/s)	…	대상 외

3,929행 → 중복 7행 지움 → approval_records_clean.csv 3,922행 → 대상 4개 댐 → 80행

지운 행은 approval_preprocess_log.csv에 (지운 번호 2220, 남긴 번호 2218, 이유)로 남긴다. 2219는 방류시작시간(18:40)이 달라 남긴다.

그림 2. 승인 기록 원본의 실제 행과 처리 — 중복 행 2220 을 지우고 대상 댐 행만 남긴다

K-water 시간자료: 두 파일을 이어 붙이고 24 시를 다음날 0 시로 읽는다

- 시간자료는 두 번에 나눠 받았다. 승인 기간에 맞춰 받은 파일(2010-08~2020-12)과 처음 받은 파일(2021-01~2026-09)은 기간이 이어져 겹치는 시각이 없으므로 그대로 이어 붙인다.
- 시각 라벨 SDATE_raw 는 YYYYMMDDHH 형식이고 시가 01~24 로 되어 있다. "24 시"는 일반 시각으로 읽을 수 없으므로 다음날 0 시로 읽는다. 그림 3 의 2020123124 가 2021-01-01 00:00 이 된다. 이렇게 해야 승인·운영 시각과 같은 시각 체계로 비교할 수 있다.
- 수위, 유입량 같은 값은 바꾸지 않고, 빈 값도 채우지 않는다. 결과는 4 개 댐 × 6 개 변수의 508,246 개 (댐, 시각)이며, KG 가 아닌 측정 저장소에 둔다.

파일 ② 승인 기간에 맞춰 추가로 받은 자료 (2010-08 ~ 2020-12)

DAM_CD	SDATE_raw	수위	유입량
1003110	2020123122	131.54	9.441
1003110	2020123123	131.55	171.565

↕ 기간이 이어진다 (겹치는 시각 없음)

파일 ① 처음 받은 자료 (2021-01 ~ 2026-09)

DAM_CD	SDATE_raw	수위	유입량
1003110	2020123124	131.54	0
1003110	2021010101	131.55	186.52

⟹

측정 저장소 이어 붙이고 시각을 읽음

SDATE_raw	시각	수위	유입량
2020123122	2020-12-31 22:00	131.54	9.441
2020123123	2020-12-31 23:00	131.55	171.565
2020123124	2021-01-01 00:00	131.54	0
2021010101	2021-01-01 01:00	131.55	186.52

· 라벨의 시(HH)가 01~24라서 24시는 다음날 0시로 읽는다 (노란 행). 값은 바꾸지 않는다.
· 4개 댐 × 6개 변수(수위, 저수량, 강우량, 유입량, 총방류량, 저수율), 508,246개 (댐, 시각). 빈 값은 채우지 않는다.

그림 3. K-water 시간자료의 실제 행 — 두 파일의 경계와 24 시 변환 (충주댐)

연계운영규정 별표 3: XML 에서 별표 3 만 꺼낸다

- 규정 XML 에는 규정 정보, 조문 24 개, 부칙, 별표 1~3 이 태그로 나뉘어 있다(그림 4).
- 태그 <별표단위> 중 별표번호가 0003 인 것의 <별표내용>만 꺼내, 글자를 바꾸지 않고 문서 저장소에 조회 키 "별표 3"으로 저장한다. 홍수기 제한수위의 근거로 원문을 그대로 보여 주기 위해서이다.
- 원문의 문자로 그린 표에서 댐별 값을 읽는 일은 단계 4 에서 한다.

규정 XML (연계운영규정, 2026-07-08 시행)

```
<AdmRulService>
  <행정규칙기본정보> 규정명, 발령일자 …
  <조문내용> 제1조(목적) … ×24
  <부칙> …
  <별표>
    <별표단위> 별표번호 0001 연계운영 대상 시설
    <별표단위> 별표번호 0002 협의회 구성
    <별표단위> 별표번호 0003 댐의 홍수기 제한수위(제6조)
      <별표내용> [별표 3] … ┌──┬──────┬…
```

⟹

문서 저장소 kgv4_documents.csv

조회 키	원문
별표 3	댐의 홍수기 제한수위(제6조) |한 강 |다목적댐 |소양강댐 |190.3 |… |수 계 | |충 주 댐 |138.0 |…

XML 태그로 별표번호 3인 <별표단위>만 꺼내고, 글자는 바꾸지 않고 원문 그대로 저장한다.

그림 4. 규정 XML 의 구조와 별표 3 꺼내기

4. 단계 3. 구조화 자료 매핑

4.1 따른 원리: D2RQ 의 매핑 원리

- Zhou 외(2023)는 댐 안전 모니터링 온톨로지 OntoDSMS 를 만들면서, MySQL 의 모니터링 데이터를 기존 오픈소스 도구 D2RQ 로 온톨로지에 연결했다. 본 연구는 D2RQ 의 매핑 원리를 벤치마킹했다(부록 E.2).
- 원리 1. 행의 키 값으로 Entity 의 식별자를 만든다 (예: item_ID 가 7 인 행 → item/7)
- 원리 2. 한 표에서 나온 식별자는 모두 같은 Class 의 Entity 가 된다
- 원리 3. 열 하나를 Data Property 하나에 대응시키고, 셀 값을 그 값으로 옮긴다
- 원리 4. 열과 속성의 대응은 사람이 온톨로지 용어로 정한다
- 그림의 노란 점선 상자는 그 과정에 쓰인 원리와 D2RQ 구문이다. 문헌의 instance 는 우리 용어로 Entity 이다.

4.2 승인 표 한 행이 Entity 가 되는 과정

입력: 승인 표와 열의 역할 (원리 4)

- 입력: 전처리를 마친 승인 표(approval_records_clean.csv)에서 대상 4 개 댐의 80 행. 한 행은 승인 한 건이다.
- 변환하기 전에 열마다 어떤 역할을 할지 사람이 먼저 정했다. 표 3 의 머리 색이 역할이다.

표 3. 승인 표(일부)와 열의 역할

순차번호	관측소코드	관측소명	승인년월일시분	방류시작시간	접수방류량	비고
473	1006110	횡성	2011-06-24	2011-06-24 18:00	100	수문방류 승인
504	**1003110**	**충주**	**2011-06-27**	**2011-06-27 18:00**	**2000**	**(실제방류는 20 시부터)**
574	1003110	충주	2011-07-01	2011-07-01 18:00	2000	(실제 방류는 19 시부터)
식별자 키 (과정 1)	옮기지 않음 (댐 코드와 같은 값)	옮기지 않음 (코드와 같은 정보)	속성 하나 (과정 3)	세 열을 내용 속성 하나로 (과정 3)		

- 관측소코드: 자료 준비 때 승인 파일의 관측소코드와 K-water 의 댐 코드(DAM_CD)를 대조했고, 4 개 댐 모두 같은 코드였다(1003110 충주 ↔ 1003110 충주댐 등). 댐 자체의 정보는 댐 자료에서 만들므로(4.3) 승인 Entity 의 속성으로 옮기지 않는다.
- 관측소명: 관측소코드와 같은 정보라 옮기지 않는다.
- 이 밖에 원본 CSV 의 행 번호와 출처 열(기관, 내려받은 주소, 원본 파일명 등)이 있으며, 과정 4·5 에 쓴다.

과정 1. 식별자 만들기 (원리 1)

- 틀 "APR:" + {순차번호}에 504 를 넣어 승인 Entity APR:504 를 만든다.
- 같은 행에서 틀 "EVI:AP:" + {순차번호}로 그 승인이 적힌 자료의 Entity EVI:AP:504 를 만든다.
- 순차번호를 키로 쓴 이유: 원본 자료가 승인 한 건마다 매긴 번호이고, 전처리에서 중복 행을 지운 뒤에는 표 안에서 유일하다. 그래서 80 행은 서로 다른 ID 80 쌍이 된다.

- ID 는 처리 순서나 무작위 번호가 아니라 원본의 순차번호로 만든다. 그래서 같은 행은 언제 다시 만들어도 같은 ID 가 된다.



순차번호	승인 Entity ID	근거 Entity ID
473	APR:473	EVI:AP:473
504	APR:504	EVI:AP:504
574	APR:574	EVI:AP:574
⋯ 80행 → 서로 다른 ID 80쌍		

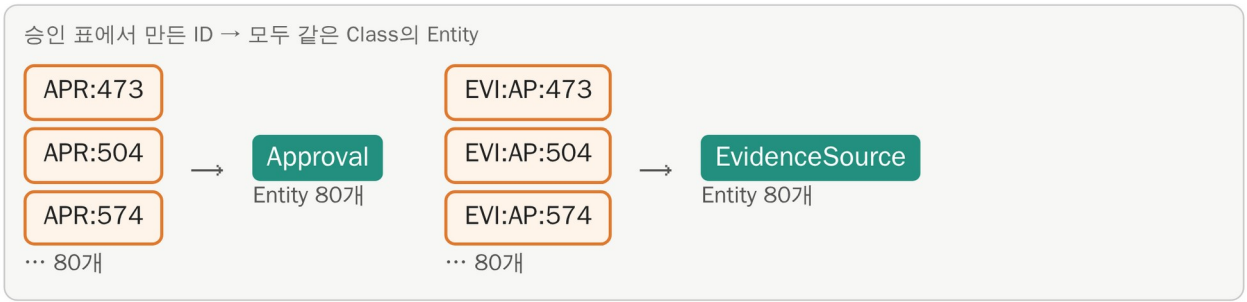
· 행 하나 → 승인 ID 하나 + 근거 ID 하나
· ID는 원본의 순차번호로 만든다
· 그래서 같은 행은 언제 다시 만들어도 같은 ID

원리 1 · D2RQ 구문 `d2rq:uriPattern "item/@@item.item_ID@@"`
@@ 사이 열(item_ID)의 값을 행마다 끼워 식별자를 만든다: item_ID = 7 → `item/7`

그림 5. 식별자 만들기 — 키 열의 값을 틀에 끼운다

과정 2. Class 지정 (원리 2)

- 승인 표에서 만든 APR: ID 80 개는 모두 Approval Class 의 Entity 로 기록한다.
- EVI:AP: ID 80 개는 모두 EvidenceSource Class 의 Entity 로 기록한다.



원리 2 · D2RQ `d2rq:class :Item` — 그 표의 모든 행 식별자를 Item Class의 Entity로 선언한다

그림 6. Class 지정

결과: CSV 파일 kgv4_entities.csv 에 줄이 생긴다 (한 줄 = Entity 하나)

Entity ID	Class
APR:504	Approval
EVI:AP:504	EvidenceSource

과정 3. 열 → Data Property (원리 3)

- 승인년월일시분 → approvalTime. 원본 값에 시각이 없어 앞 10 글자를 날짜로 저장한다(column).

- 방류시작시간, 접수방류량, 비고 → approvalContent 하나. 세 값을 글자 틀에 넣어 저장한다(pattern). 비고는 원문 그대로 넣는다.
- 세 열을 합친 이유: 주어진 온톨로지에 Approval 의 내용 속성이 approvalContent 하나뿐이었다(7 장 검토할 점 1).

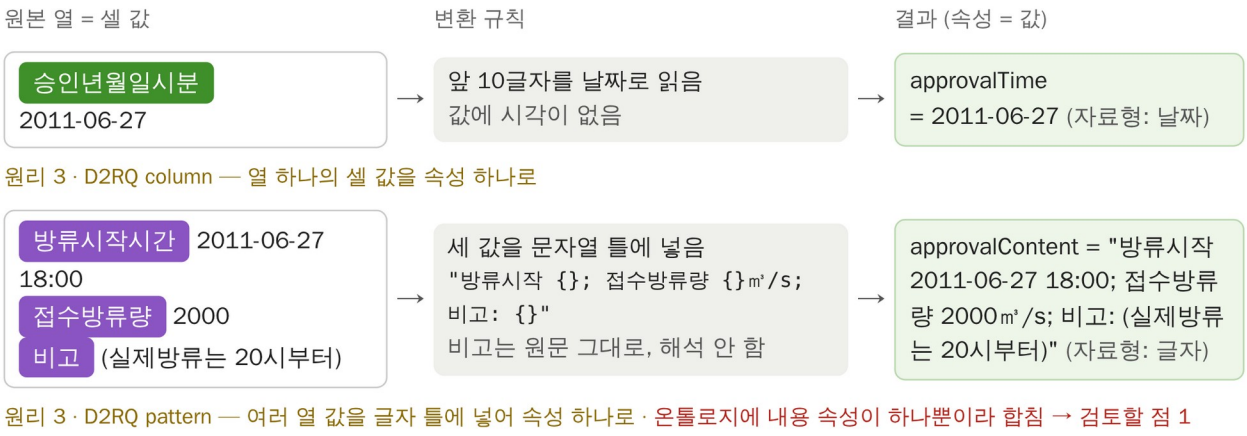


그림 7. 열 → Data Property — 원본 셀 값, 변환 규칙, 결과

결과: CSV 파일 kgv4_properties.csv 에 줄이 생긴다 (한 줄 = Entity 의 속성 하나)

Entity ID	속성	값	자료형
APR:504	approvalTime	2011-06-27	날짜
APR:504	approvalContent	방류시작 2011-06-27 18:00; 접수방류량 2000㎥/s; 비고: (실제방류는 20 시부터)	글자

과정 4. 근거 Entity 와 연결

- 같은 행의 출처 열을 근거 Entity EVI:AP:504 의 속성으로 옮긴다(아래 표).
- APR:504 와 EVI:AP:504 는 같은 504 행을 처리하면서 동시에 만든다. 그래서 만드는 그 자리에서 APR:504 —supportedBy→ EVI:AP:504 로 잇는다(그림 8).
- 두 ID 의 504 는 둘이 같은 원본 행에서 나왔다는 표시이다. 나중에 ID 글자(AP, APR)를 맞춰 보고 찾아서 잇는 것이 아니다.



그림 8. 같은 행에서 함께 만들며 잇는다 (APR:504 → EVI:AP:504)

결과: 근거 Entity 의 속성도 같은 CSV 파일 kgv4_properties.csv 에 줄로 추가된다

Entity ID	속성	값	자료형
EVI:AP:504	sourceTitle	환경부 한강홍수통제소 홍수예보 댐방류승인	글자
EVI:AP:504	sourceType	승인자료	글자
EVI:AP:504	sourceLocator	순차번호 504	글자

과정 5. 출처 기록

- 우리 원본은 DB 가 아니라 내려받은 CSV·JSON 파일이어서, 매핑한 값을 옮겨 저장한다.
- 값을 옮기면 KG 의 값과 원본 행의 연결이 끊긴다. 그래서 Entity 마다 원본 파일, 그 안의 위치, 적용한 규칙, 값의 성격을 출처 파일에 남긴다.
- 위치(504 번째 행)로 원 파일의 해당 행을 바로 열 수 있다. 값의 성격 "원본 그대로"는 원본 값을 가공 없이 옮겼다는 뜻이다.

결과: CSV 파일 kgv4_provenance.csv 에 줄이 생긴다 (한 줄 = Entity 의 출처 하나)

Entity ID	원본 파일	위치	규칙	값의 성격	메모
APR:504	approval_records_clean.csv	504 번째 행	MR-APR	원본 그대로	승인일은 날짜만 있음

결과: CSV 파일 3 개

- 단계 3 의 결과물은 CSV 파일 3 개이며, 03_normalized 폴더에 저장된다.
- kgv4_entities.csv: Entity ID 와 Class (한 줄 = Entity 하나)
- kgv4_properties.csv: Entity ID, 속성, 값, 자료형 (한 줄 = Entity 의 속성 하나)
- kgv4_provenance.csv: Entity ID, 원본 파일, 위치, 규칙, 값의 성격 (한 줄 = Entity 의 출처 하나)
- 세 파일의 줄을 Entity ID(APR:504)로 모으면 Entity 하나의 전체 모습이 된다: Class, 속성값, 출처(그림 9).

파일 안의 실제 모습 (첫 줄은 열 이름, 이어서 APR:504 의 줄)

```
kgv4_entities.csv
  entity_id,class,label
  APR:504,Approval,승인 504 (충주댐)

kgv4_properties.csv
  entity_id,property,value,datatype
  APR:504,approvalTime,2011-06-27,date
  APR:504,approvalContent,방류시작 2011-06-27 18:00; 접수방류량 2000㎥/s; 비고: (실제방류는 20
시부터),string

kgv4_provenance.csv
  item,kind,source_file,source_locator,rule,data_origin,note
  APR:504,entity,03_normalized/approval_records_clean.csv,row_number_in_file=504,MR-APR,real,
승인일은 날짜만 있음
```

kgv4_entities.csv 한 줄 = Entity 하나 (일부 열)

Entity ID	Class
APR:504	Approval

kgv4_properties.csv 한 줄 = Entity의 속성 하나

Entity ID	속성	값	자료형
APR:504	approvalTime	2011-06-27	날짜
APR:504	approvalContent	방류시작 2011-06-27 18:00; 접수방류량 2000㎥/s; 비고: (실제방류는 20시부터)	글자

kgv4_provenance.csv 한 줄 = Entity의 출처 하나

Entity ID	원본 파일	위치	규칙	값의 성격
APR:504	approval_records_clean.csv	504번째 행	MR-APR	원본 그대로

→

CSV 3개의 줄을 Entity ID로 모은 모습

APR:504 Approval

approvalTime = 2011-06-27 (날짜)

approvalContent = "방류시작 2011-06-27 18:00; 접수방류량 2000㎥/s; 비고: (실제방류는 20시부터)" (글자)

출처: approval_records_clean.csv 504번째 행 · 규칙 MR-APR · 원본 값 그대로

그림 9. CSV 파일 3 개의 줄과 그것을 모은 Entity (APR:504)

4.3 댐 자료와 측정 자료

Dam (댐 기본 자료)

- 입력: 댐 한 개가 한 행인 표는 따로 없다. K-water 일자료에는 댐 하나의 하루 측정값이 한 행이고, 행마다 댐 코드(DAM_CD), 댐 이름, 댐 유형이 반복해서 붙어 있다.
- 식별자: 틀 "DAM:" + {DAM_CD}로 만든다. 충주댐의 12,582 행은 모두 DAM_CD 가 1003110 이므로 모두 DAM:1003110 이 되고, Entity 하나로 모인다(원리 1).
- Class: Dam. 속성: 댐 유형 열 → damType(다목적댐). damName 에는 승인 자료의 "충주"와 맞춘 표준 댐명(충주댐)을 넣는다.
- 결과: 승인과 같은 CSV 파일 3 개에 Dam Entity 의 줄이 추가된다.

K-water 일자료 (한 행 = 댐 하나의 하루 측정값 하나)

DAM_CD	댐 이름	댐 유형	날짜	변수	값
1003110	충주댐	다목적댐	2021-12-31	저수위	131.53
1003110	충주댐	다목적댐	2021-12-30	저수위	131.63
1003110	충주댐	다목적댐	2021-12-29	저수위	131.71
⋯ DAM_CD 1003110인 행 12,582개					

DAM_CD = 1003110 (12,582행 모두) → `"DAM:" + {DAM_CD}` → DAM:1003110 ID 하나 → Entity 하나

DAM:1003110 Dam

damName = 충주댐 (승인 자료의 "충주"와 맞춘 표준 댐명)

damType = 다목적댐 (댐 유형 열의 값)

원리 1 적용 — ID를 키 값(DAM_CD)으로 만들기 때문에, 같은 댐의 여러 행은 모두 같은 ID가 되어 Entity 하나로 모인다

그림 10. K-water 일자료 → Dam Entity

HydrometeorologicalState (HMS, 수문·기상 상태)

- 입력: K-water 시간자료. 한 행이 (댐 코드, 시각)이고 변수 6 개(수위, 저수량, 강우량, 유입량, 총방류량, 저수율)가 열로 펼쳐진 넓은 형태이며 553,609 행이다.
- 행마다 Entity 를 만들면 55 만 개가 된다. KG 에 필요한 것은 "어느 댐에 어떤 측정 변수가 있는가"이므로, 키 열(DAM_CD)과 열 머리(변수명)를 함께 틀에 넣어 (댐, 변수)마다 Entity 를 만든다. 예: HMS:1003110:수위
- 결과: 댐 4 × 변수 6 = 24 개. 속성은 variableType(수위 등)과 stationCode(1003110)이다. 시각별 값은 KG 에 넣지 않고 측정 저장소에 그대로 둔다.
- 같은 자료에서 댐마다 측정자료 근거 Entity EVI:KW:{DAM_CD}를 만들고, HMS 와 supportedBy 로 잇는다.

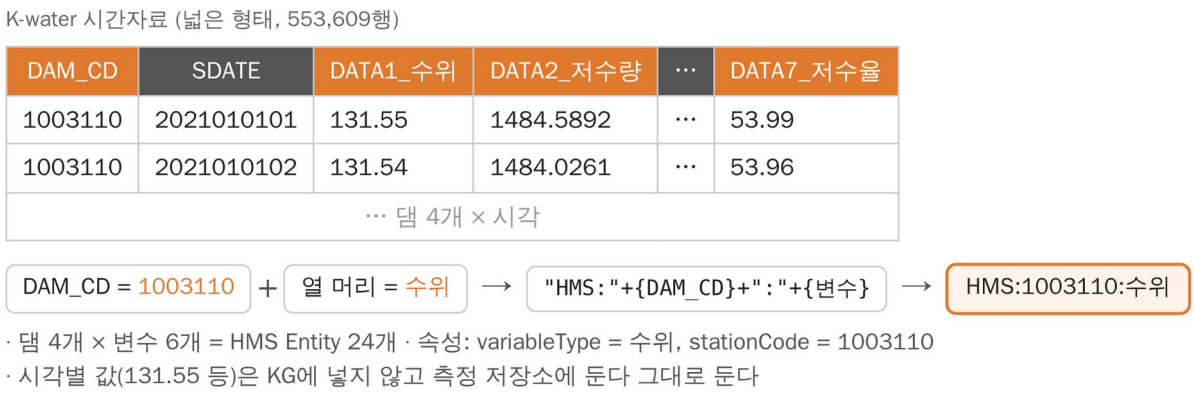
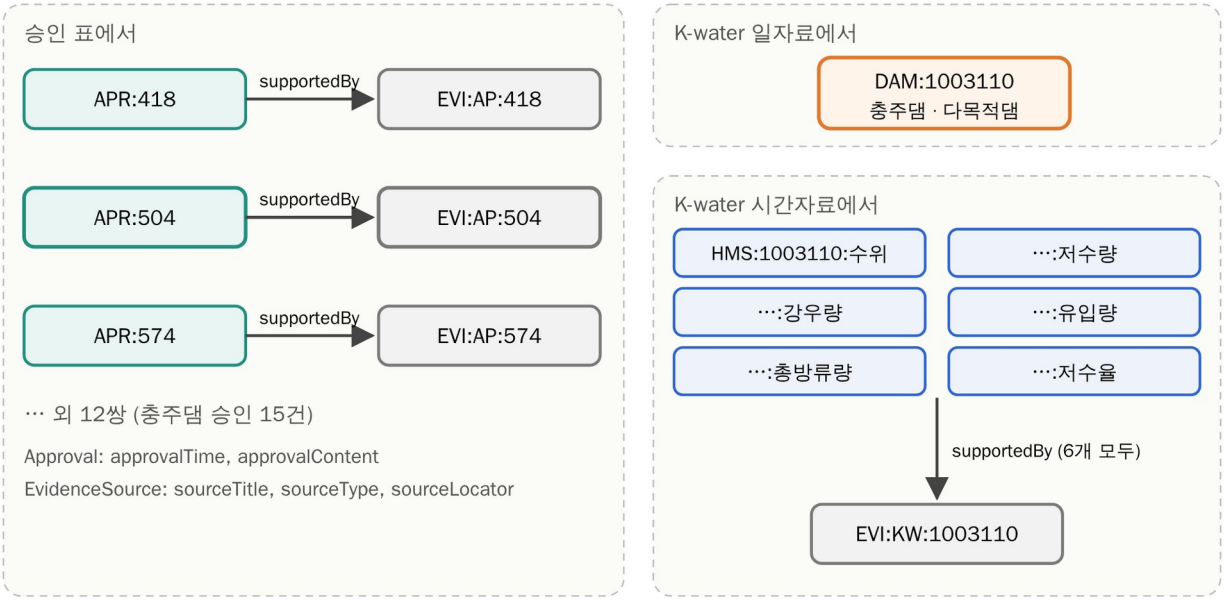


그림 11. K-water 시간자료 → HMS Entity

4.4 단계 3 이 끝난 상태

- 그림 12 는 충주댐에 관련된 Entity 를 만든 자료별로 모은 것이다.
- 각 자료에서 Entity 와 속성이 만들어졌고, 같은 행·같은 자료에서 함께 만든 근거만 supportedBy 로 이어져 있다.
- 서로 다른 자료에서 만든 승인, 댐, 측정 상태 Entity 는 아직 서로 떨어져 있다. 이것을 단계 5 에서 잇는다.
- 4 개 댐 전체: Approval 80, EvidenceSource 84(승인자료 80, 측정자료 4), HMS 24, Dam 4. 모두 Entity 192 개, 속성 468 개.



각 자료에서 Entity와 속성이 만들어졌고, 같은 행·같은 자료에서 함께 만든 근거만 supportedBy로 이어져 있다(실선). 서로 다른 자료에서 만든 승인, 댐, 측정 상태 Entity는 아직 서로 떨어져 있다.

그림 12. 단계 3 이 끝난 상태 — 충주댐의 Entity

5. 단계 4. 비정형 텍스트 추출

5.1 따른 원리: DDKG 의 규칙 기반 지식 추출

- DDKG 의 안전평가 결과(예: "홍수 방어 안전 등급은 C 등급이다")는 표준화된 문형으로 쓰이므로 딥러닝 없이 정규식으로 바로 뽑는다. 표현이 제각각인 위험 서술에만 LLM 을 쓴다(부록 E.1).
- 우리 자료는 규칙 쪽에 해당한다. 비고는 한 기관이 쓰는 짧은 문구라 어휘가 한정되어 있고("탄력적 방류", "초기수문방류", "실제방류는 N 시부터"), 별표 3 은 칸 배치가 고정된 표이다. 그래서 LLM 추출은 쓰지 않았다.

5.2 비고 → Operation (운영행위)

- 대상: 단계 3 에서 승인 Entity 에 원문 그대로 옮긴 비고. 80 개 승인의 비고를 한 건씩 아래 순서로 처리한다.
- ① 행위가 아닌 조건 문구를 지운다: "발전방류 포함", "여수로 자연월류량 제외" 등
- ② 승인 변경 문구("변경", "연장", "정정")가 있으면 새 방류가 아니라 같은 방류의 승인을 바꾼 것으로 보고 Operation 을 만들지 않는다. 만들지 않은 사유는 기록한다.
- ③ 행위 문구 규칙(표 4)을 대조해, 잡힌 문구마다 Operation 하나를 만든다. 한 비고에서 두 개가 잡히면 두 개를 만든다.
- ④ 행위 문구가 없으면 일반 "방류" 하나를 만든다(실제 방류 시각 문구, 일반 방류 문구, 비고 없음).
- ⑤ 시각: 비고에 시각이 적혀 있으면 그 시각, 없으면 승인의 방류시작시간을 쓴다.
- ID 는 "OPR:" + {순차번호} + ":" + {순번}으로 만든다. 한 승인에서 Operation 이 둘이면 OPR:2754:1, OPR:2754:2 가 된다.

표 4. 행위 문구 규칙과 적용 건수 (대상 80 건)

operationType	찾는 문구 (정규식)	예	건수
방류종료	방류\s*종료	9.11 16:00 수문방류종료	1
초기방류	초기\s*수문\s*방류 | 초기\s*방류 | \(\s*초기\s*\d	(초기수문방류 100㎥/s)	9
점진·점증방류	점진 | 점증	이내 점증방류	1
증가방류	증가\s*방류 | \d\s*cms\s*증 | ㎥/s\s*증가 등	(증가방류 200 ㎥/s)	7
감소방류	감소\s*방류	—	0
탄력적 방류	탄력	최대 100㎥/s 이내 탄력적 방류	25
방류	실제\s*방류는\s*(\d+)\s*시부터 / 수문\s*방류 등 / 비고 없음	(실제방류는 20 시부터)	25
(만들지 않음)	변경 | 연장 | 정정	방류기간 (당초) … (변경) …	17 건

승인 (방류시작)	비고 원문	적용한 규칙	Operation
504 충주 18:00	(실제방류는 20시부터)	행위 문구 없음 → 실제 방류 시각 문구 → 방류, 시각은 비고의 20시	OPR:504:1 방류 · 20:00
2754 소양강 14:00	최대 1500㎥/s~~(발전방류 포함)~~이내 탄력적 방류 (초기 1000㎥/s이내 방류)	조건 문구 지움 → 행위 문구 2개 (초기, 탄력) → Operation 2개, 시각은 방류시작시간	OPR:2754:1 초기방류 OPR:2754:2 탄력적 방류 · 14:00
279 횡성 17:00	9.11 16:00 수문방류종료	행위 문구(종료) → 방류종료, 시각은 비고의 9.11 16:00	OPR:279:1 방류종료 · 09-11 16:00
418 충주 15:00	(비어 있음)	비고 없음 → 승인 자체를 방류로, 시각은 방류시작시간	OPR:418:1 방류 · 15:00
3519 충주 08-02 18:00	방류기간 (당초) 08/02 18:00 ~ 08/12 18:00 (변경) 08/02 18:00 ~ 08/16 24:00 …	승인 변경 문구 → 같은 방류의 변경으로 보고 만들지 않음 (사유 기록)	만들지 않음

~~회색 취소선~~ 지운 조건 문구 · 보라 행위 문구 · 주황 비고에 적힌 시각 · 빨강 승인 변경 문구

그림 13. 실제 비고 5 건이 Operation 이 되는 과정

결과: 중간 파일 operation_from_remarks_5dams.csv(만든 68 행 + 만들지 않은 17 행과 사유), 그리고 KG CSV 파일에 Operation 의 줄

Entity ID	속성	값	자료형
OPR:504:1	operationType	방류	글자
OPR:504:1	operationTime	2011-06-27T20:00	일시
OPR:2754:1	operationType	초기방류	글자
OPR:2754:2	operationType	탄력적 방류	글자

- 80 개 승인 중 63 개에서 Operation 68 개가 만들어진다(58 개 승인에서 1 개, 5 개 승인에서 2 개). 17 개 승인은 변경 승인이라 만들지 않았다.
- Operation 도 승인과 같은 행에서 나왔으므로, 근거 Entity EVI:AP:{순차번호}에 supportedBy 로 바로 잇는다.

5.3 별표 3 → Criterion (홍수기 제한수위)

- 대상: 단계 2 에서 꺼내 둔 별표 3 원문. 선 문자로 그린 표라 칸이 "|"로 나뉘어 있고, 댐 이름 글자 사이에 공백이 있다("충 주 댐").
- ① 댐 이름마다 정규식으로 그 댐의 행을 찾는다. 이름 글자 사이에 공백을 허용(\s*)해서 "충 주 댐"과 "충주댐"을 같은 이름으로 읽고, 이어지는 두 칸(홍수기 제한수위, 시설별 최저 운영수위)의 글자를 원문 그대로 남긴다(criterion_list.csv).
- ② 대상 댐이면서 제한수위 칸이 숫자뿐인 경우에만 Criterion 을 만든다. 값은 숫자(criterionValue), 단위는 EL.m 이다.
- 결과: 충주, 소양강, 횡성 3 개(CRI:CR-01~03). 광동댐은 별표 3 에 없다. Criterion 은 규정 근거 EVI:LAW: 별표 3 에 supportedBy 로 잇는다.

문서 저장소의 별표3 원문 (일부)

```
|구 분 |댐 종류    |댐 명      |홍수기 제한수위 |시설별 최저 운영수위|
|한 강 |다목적댐   |소양강댐   |190.3           |-                   |
|수 계 |           |충 주 댐   |138.0           |-                   |
|       |           |횡 성 댐   |178.2           |-                   |
|       |           |팔 당 댐   |25.51)          |-                   |
```

댐 이름 "충주댐" → `|\s*충\s*주\s*댐\s*|\s*(값)\s*|\s*(값)\s*|` → 칸 원문 '138.0', '-'

글자 사이 공백을 허용(\s*)해서 "충 주 댐"과 "충주댐"을 같은 이름으로 읽는다. 칸의 글자는 해석하지 않고 원문 그대로 남긴다.

댐	제한수위 칸 원문	Criterion	criterionValue	unit
소양강댐	'190.3'	CRI:CR-01	190.3 (숫자)	EL.m
충주댐	'138.0'	CRI:CR-02	138.0 (숫자)	EL.m
횡성댐	'178.2'	CRI:CR-03	178.2 (숫자)	EL.m
팔당댐	'25.51)' (값 25.5 + 각주 1)	대상 댐 아님 · 각주가 붙은 값은 숫자만 받는 규칙에서 빠짐		

그림 14. 별표 3 의 문자로 그린 표에서 댐별 제한수위 칸 읽기

6. 단계 5. 개체 해소·연결과 구축 결과

6.1 따른 원리: GraphAide 의 개체 중의성 해소

- GraphAide(Purohit 외 2024)는 뉴스 기사에서 뽑은 이름(mention)을 기준 지식베이스(DARPA 판 Wikidata)의 고유번호(QID)에 대응시켜, 다르게 적힌 이름도 같은 Entity 로 묶는다(부록 E.3).
- 원리는 "자료마다 다르게 적힌 대상을 기준 목록의 고유 식별자 하나에 대응시키고, 그 식별자로 잇는다" 이다.
- 우리 적용: 기준 식별자는 K-water 댐 코드(DAM_CD)이다. 같은 충주댐이 승인 자료에는 "충주"(코드 1003110), K-water 에는 "충주댐", 별표 3 에는 "충 주 댐"(코드 없음)으로 적혀 있다.
- GraphAide 는 기사 속 이름이 애매해서 임베딩 검색과 LLM 으로 가장 가까운 QID 를 고른다. 우리 자료에는 코드나 정해진 이름이 있어서, LLM 없이 기준 목록에서 바로 찾는다.

6.2 연결 과정

- ① 기준 목록을 만든다: 단계 3 에서 만든 Dam Entity 4 개와 DAM_CD, 표준 댐명, 자료별 표기(그림 15 위)
- ② Entity 마다 끝점을 찾는 키로 기준 목록을 찾는다. 코드가 있으면 코드로(승인의 관측소코드, HMS 의 stationCode), 코드가 없으면 공백을 지운 이름으로(별표 3 의 댐 이름) 찾는다.
- ③ 찾은 Dam ID 로 관계 한 줄을 관계 파일에 적는다: 시작 ID, 관계, 끝 ID, 적용한 규칙, 값의 성격
- ④ 같은 승인 행에서 만든 Entity 끼리(승인과 운영행위)는 찾을 필요 없이 바로 잇는다. 운영행위의 댐은 그 승인 행의 관측소코드로 정한다.
- ⑤ 기준 목록에서 찾지 못하면 잇지 않는다. 추측으로 연결하지 않는다.

기준 목록 — Dam Entity 4개 (K-water 댐 코드 DAM_CD가 기준 식별자)

DAM_CD	Dam Entity	표준 댐명	승인 자료의 관측소명	별표3의 표기
1003110	DAM:1003110	충주댐	충주	충 주 댐
1012110	DAM:1012110	소양강댐	소양강	소양강댐
1006110	DAM:1006110	횡성댐	횡성	횡 성 댐
1001210	DAM:1001210	광동댐	광동	(없음)

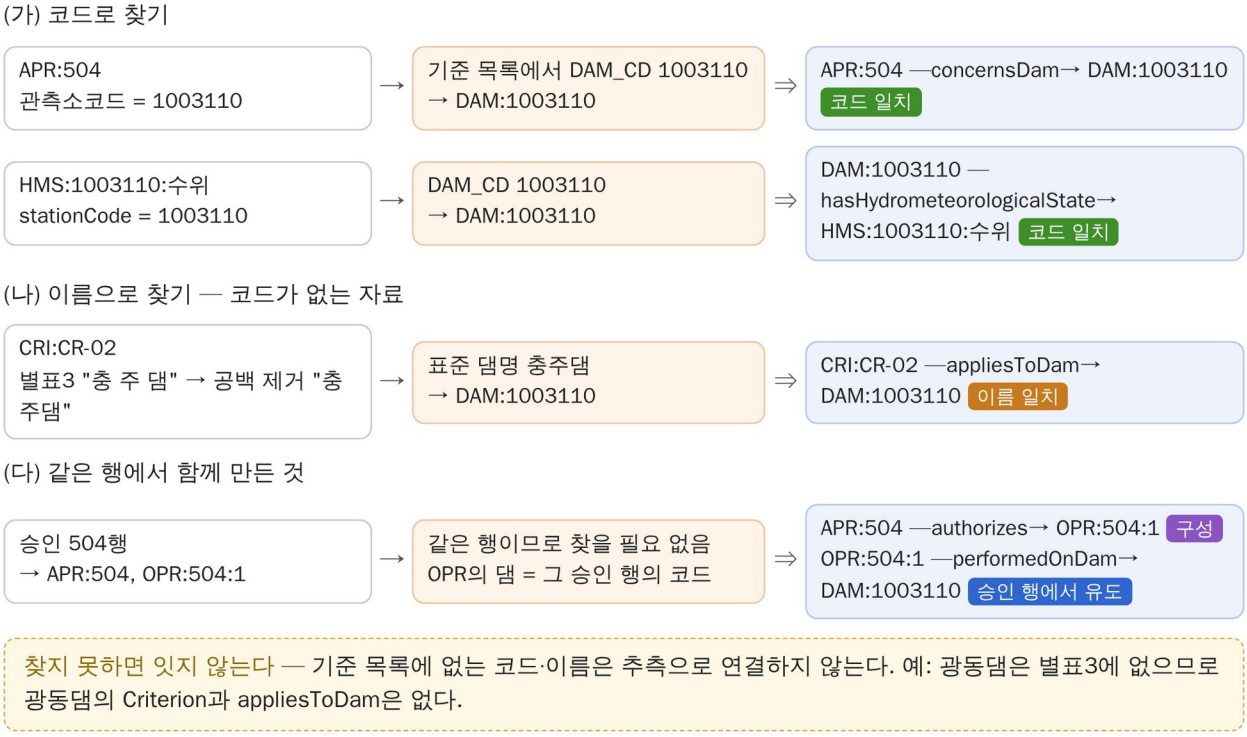


그림 15. 기준 목록과 연결 방법 — 코드로, 이름으로, 같은 행에서

표 5. 관계별 연결 방법과 결과

관계	시작 → 끝	끝을 찾는 방법	값의 성격	건수
concernsDam	Approval → Dam	승인의 관측소코드 = DAM_CD	실제 코드 일치	80
hasHydrometeorologicalState	Dam → HMS	DAM_CD = HMS 의 stationCode	실제 코드 일치	24
appliesToDam	Criterion → Dam	별표 3 댐 이름(공백 제거) = 표준 댐명	이름 일치	3
authorizes	Approval → Operation	같은 승인 행에서 만든 Operation	구성(실행 확인 아님)	68
performedOnDam	Operation → Dam	Operation 을 만든 승인 행의 관측소코드	승인 행에서 유도	68
supportedBy	각 Entity → EvidenceSource	같은 행·같은 자료의 근거 (단계 3·4 에서 함께 만듦)	실제 / 유도	175
합계				418

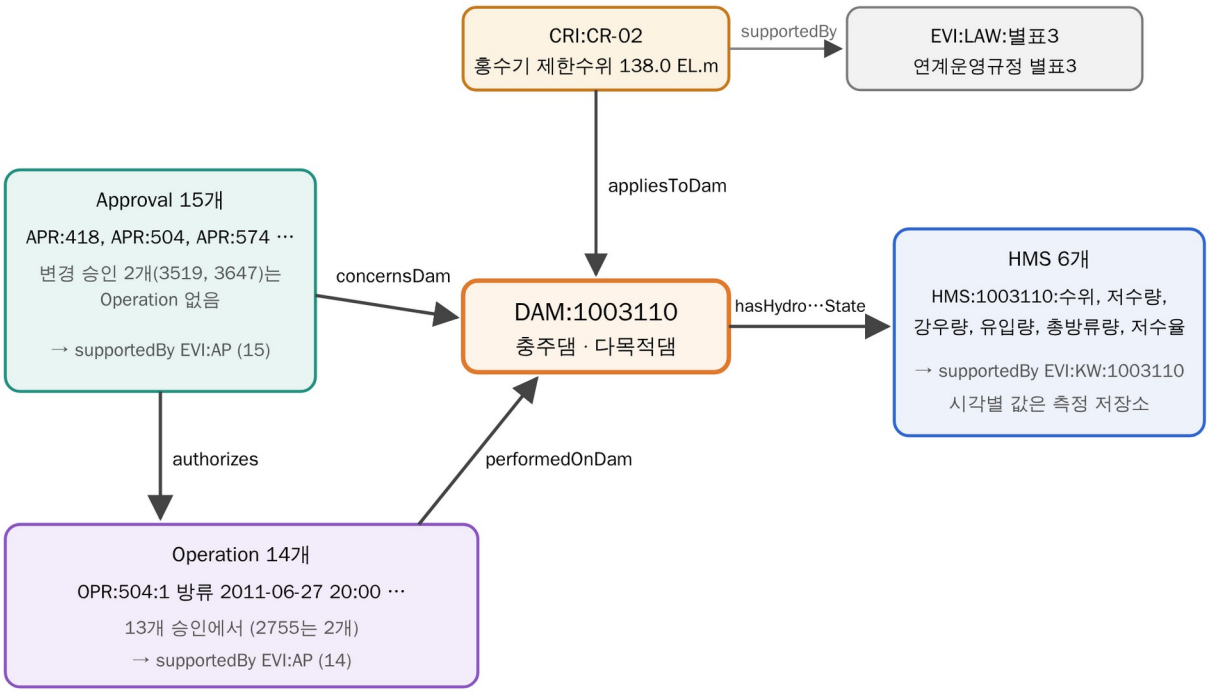
결과: CSV 파일 kgv4_relations.csv 에 줄이 생긴다 (한 줄 = 관계 하나)

```
kgv4_relations.csv
  subject,relation,object,rule,origin,note
  APR:504,concernsDam,DAM:1003110,MR-APR-DAM,real,승인 레코드의 관측소코드 = 댐 코드(DAM_CD)
  APR:504,authorizes,OPR:504:1,MR-AUTH,constructed,같은 승인 행의 비고에서 분류한 행위
  OPR:504:1,performedOnDam,DAM:1003110,MR-OPR-DAM,derived,승인 행의 시설 코드
  CRI:CR-02,appliesToDam,DAM:1003110,MR-CRI-DAM,name-match,별표 3 시설명 일치(코드 없음)
```

- origin 열은 그 관계를 얼마나 믿을 수 있는지 나타낸다. 관계 418 개 중 실제 코드나 원본 그대로인 것(real) 211, 원본에서 유도한 것(derived) 136, 연구용으로 구성한 것(constructed) 68, 이름 일치(name-match) 3 이다.
- 10/16 자료에서는 변경 승인 17 건 중 11 건을 "같은 댐, 같은 방류 시작 시각"으로 원 승인의 운영행위에 추정 연결했다(12 개 관계). 추측 연결을 하지 않는다는 원칙에 맞지 않아 없앴다(7 장 검토할 점 6).

6.3 구축 결과: 완성된 KG

- 단계 5 가 끝나면 충주댐 Entity 를 중심으로 승인 15 개, 운영행위 14 개, 측정 상태 6 개, 운영기준 1 개와 각각의 근거가 하나의 그래프로 이어진다(그림 16).
- 4 개 댐 전체: Entity 264 개(Approval 80, EvidenceSource 85, Operation 68, HMS 24, Dam 4, Criterion 3), 속성 616 개, 관계 418 개.
- KG 는 CSV 파일 4 개(Entity, 속성, 관계, 출처)로 이루어진다. 이 파일을 검증하고(단계 6) 그래프 DB 에 저장하는 일(단계 7)은 다음 주에 다룬다.



단계 5가 끝나면 충주댐 Entity를 중심으로 승인, 운영행위, 측정 상태, 운영기준과 각각의 근거가 하나의 그래프로 이어진다.

그림 16. 단계 5 가 끝난 KG — 충주댐을 중심으로 이어진 Entity 와 관계

7. 검토할 점

1. approvalContent 하나에 세 열이 들어가 있음 (단계 3, 온톨로지 발전 후보)

- 현재: 방류시작시간, 접수방류량, 비고를 글자 하나로 저장한다. 접수일자 열은 어느 속성에도 들어가지 않았다.
- 이유: 구축 당시 주어진 온톨로지에서 Approval 의 Data Property 는 approvalTime 과 approvalContent 두 개뿐이었고, 온톨로지는 수정하지 않는다는 전제로 매핑했다.
- 원리 3 대로라면 속성도 따로여야 한다. 합친 결과 "접수방류량 2000㎥/s 이상인 승인", "승인일과 방류 시작의 시간 차" 같은 조건 질의를 할 수 없다.
- 선택지: ① Approval 에 dischargeStartTime(일시), requestedDischarge(숫자, ㎥/s), remark(글자), 필요하면 receiptDate(날짜)를 추가 ② 현재대로 두고 승인 내용은 답변에 원문으로만 보여 줌
- 판단 기준: 지금의 CQ4 는 댐과 기간으로 승인을 찾아 내용을 그대로 돌려주므로 분리가 꼭 필요하지 않다. 방류량이나 시각을 조건으로 쓰는 질문을 CQ 에 넣을 때 함께 정한다.

2. ID 접두어 표기 (단계 3, 10/19 질문: AP 와 APR)

- 연결: APR:504 와 EVI:AP:504 는 같은 504 행에서 동시에 만들어지고 그 자리에서 이어진다(과정 4, 그림 8). 그래서 글자가 달라도 연결에는 문제가 없다.
- 표기: APR 은 승인(Approval), EVI 는 근거 자료(EvidenceSource)이고, EVI 뒤의 AP 는 출처 종류(승인자료)이다. 측정자료는 KW, 규정은 LAW 이다.
- 검토: 사람이 볼 때 AP 와 APR 이 비슷해 헷갈린다. 출처 종류 표기를 Class 접두어와 맞출지(EVI:APR:504) 정한다.

3. 전처리에서 남은 확인 사항 (단계 2)

- 중복 판단 기준: 6 개 값(관측소코드, 승인일, 방류시작시간, 접수방류량, 접수일자, 비고)이 모두 같으면 같은 승인이 두 번 입력된 것으로 보았다. 실제로 같은 내용의 승인이 두 번 있었을 가능성은 원 승인 대장 없이는 확인할 수 없다.
- 24 시 라벨의 의미: K-water 시간자료의 "24 시"가 그 시각의 순간값인지 앞 1 시간의 값인지 자료에 설명이 없다. 그래서 시각이 ±1 시간 어긋날 수 있다.
- 대상 4 개 댐: 승인 자료와 측정자료의 코드가 같은 댐이라는 이유는 있지만, 같은 조건의 시설(충주조정지 등)을 왜 뺐는지는 기록되어 있지 않아 정리가 필요하다.

4. 승인 조건과 실행 정보를 구분하지 않음 (단계 4)

- 비고에는 승인 조건(허가한 범위와 방식, 예: "최대 X 이내 탄력적 방류"), 승인 변경, 실제 실행 정보(예: "실제방류는 20 시부터", "수문방류종료")가 섞여 있다.
- 지금 규칙은 승인 변경만 구분하고, 승인 조건도 운영행위로 만든다. "탄력적 방류" 25 건은 허가한 방식이고, 비고가 빈 15 건은 승인 자체를 방류로 만든 것이다.
- 그래서 Operation 은 "실제로 한 행위"가 아니라 "승인이 허가한 행위"에 가깝다(코드에도 "실행 확인 아님"으로 표시). 비고를 승인 조건 / 승인 변경 / 실행 정보로 나누고, 승인 조건은 Approval 의 속성으로 옮길지 검토한다(검토할 점 1 과 연결).

5. 별표 3 각주가 붙은 값을 건너뜀 (단계 4)

- 팔당댐의 "25.51)"처럼 값 뒤에 각주 번호가 붙으면 숫자만 받는 규칙에서 빠진다. 대상 3 개 댐에는 각주가 없어 지금은 영향이 없지만, 대상을 넓히면 값과 각주를 나누어 저장해야 한다.

6. 변경 승인 17 건은 운영행위와 이어지지 않음 (단계 5)

- 변경 승인은 Operation 을 만들지 않고, 원 승인에 추정 연결하던 것도 없앴다. 그래서 변경 승인은 댐(concernsDam)과 근거에만 이어져 있다.
- 변경 승인 비고에는 당초·변경 기간이 적혀 있다("방류기간 (당초) 08/02 18:00 ~ … (변경) …"). 이 기간으로 원 승인을 찾는 규칙을 둘지, 변경 승인과 원 승인 사이의 관계를 온톨로지에 둘지 검토한다.

7. 코드가 같으면 같은 시설인가 (단계 5)

- 승인 자료의 관측소코드와 K-water 의 DAM_CD 는 4 개 댐 모두 같고 이름도 대응하지만, 두 기관의 코드가 같은 시설을 가리킨다는 공식 확인은 없다(대조표에 "동일시설 여부 검토 필요"로 기록).

8. 다음 주에 다룰 것

단계 6. 검증 (10/19 지적: 검증 파트가 가장 약함, 다시 고민)

- 제약 하나하나의 정의와 그 제약을 고른 근거(문헌), 판정 방법과 결과
- 온톨로지를 검증하는 것인지 KG 를 검증하는 것인지, 매핑·입력 오류(KG 오류)와 온톨로지 오류의 구분

단계 7. 저장

- 저장된 형태(CSV 파일 4 개와 그 줄), Cypher 가 무엇인지, CSV 한 줄이 Cypher 한 줄을 거쳐 Neo4j 에 들어가는 모습

검색과 답변

- 의도별 경로를 미리 정해 두는 방식의 한계, 어디까지 탐색할지를 LLM 이 판단하게 하는 방안, LLM 입력 구성, CQ 를 "판단에 필요한 정보와의 관계"로 다시 정의하는 것

부록 A. 용어 정리

용어	본 연구에서의 정의
Ontology	댐 운영 분야에 어떤 종류의 대상이 있고 어떤 관계·속성으로 표현하는지 정리한 의미 구조. 실제 운영데이터는 포함하지 않는다.
Class	Ontology 가 표현하는 대상의 종류. 예: Dam, Operation, Approval.
Entity	Class 에 속하는 실제 개체. 예: 충주댐은 Dam Class 의 Entity. 문헌의 instance(인스턴스)에 해당한다.
Property	Entity 를 기술하는 방법. Entity 와 Entity 를 연결하는 Relation(Object Property)과, Entity 에 값을 부여하는 Data Property 의 두 종류로 구분한다.
Relation (Object Property)	Property 중 대상과 대상 사이의 의미적 연결을 표현하는 관계. Ontology 에서는 어떤 종류의 대상끼리 어떤 관계를 가질 수 있는지 정의하고, 실제 KG 에서는 해당 Entity 사이에 그 관계가 적용된다.
Data Property	Property 중 대상이 가질 수 있는 값을 표현하는 속성. Ontology 에서는 각 Class 가 어떤 값을 가질 수 있는지 정의하고, 실제 KG 에서는 Entity 에 구체적인 값이 채워진다.
Knowledge Graph (KG)	Ontology 에 실제 Entity·Relation·값을 채워 만든 지식 구조. 본 연구에서는 모든 원자료를 담는 데이터베이스가 아니라, 핵심 의미·관계·근거와 외부 자료를 연결하는 중심 구조로 사용한다.
Competency Question (CQ)	Ontology 가 답할 수 있어야 하는 질문으로, 연구 범위와 필요한 지식 구조를 정하는 기준.
Hybrid Retrieval	질문에 따라 KG, 문서, 측정자료 등 서로 다른 출처에서 필요한 근거를 검색하여 결합하는 방식.
정규식	글자의 모양을 패턴으로 적어 문장에서 맞는 부분을 찾는 표현. 예: "실제\s*방류는\s*(\d+)\s*시부터"는 "실제방류는 20 시부터"에서 20 을 찾는다.

부록 B. 연구 목적과 Ontology 가 답해야 할 질문

연구 목적	현재 댐의 수문·기상 상태와 관련된 과거 운영행위, 승인, 운영기준 및 근거자료를 연결하여 댐 운영자의 의사결정을 지원한다.

전체 질문	특정 댐의 현재 수문·기상 상태와 현행 운영기준을 확인하고, 선택된 과거 시점 또는 사례의 수문·기상 상태, 실제 운영행위, 방류 승인내용 및 관련 근거자료를 함께 추적하여 현재 운영 판단을 지원할 수 있는가?

번호	질문 대상	세부 질문
CQ1	현재 수문·기상 상태	대상 댐의 현재 또는 기준시점의 강우, 수위, 유입량, 방류량 등 수문·기상 상태는 무엇인가?
CQ2	과거 수문·기상 상태	선택된 과거 시점 또는 사례에서 대상 댐의 강우, 수위, 유입량, 방류량 등 수문·기상 상태는 어떠했는가?
CQ3	과거 운영행위	해당 과거 시점 또는 사례에서 실제로 수행된 댐 운영행위는 무엇이었는가?
CQ4	과거 방류 승인	해당 과거 시점 또는 사례에서 어떤 방류 승인이 있었으며, 그 승인내용은 무엇인가?
CQ5	현재 운영기준	현재 대상 댐에 적용되는 운영기준은 무엇이며, 현재 운영 판단과 관련된 기준값은 무엇인가?
CQ6	공식 근거	각 수문·기상 상태, 운영행위, 승인내용 및 운영기준을 확인할 수 있는 공식 자료 또는 근거문서는 무엇인가?

부록 C. 온톨로지 설계

Class (6 개)

Class	의미	예	표현하는 CQ
Dam	댐 운영 의사결정의 대상이 되는 물리적 시설	충주댐, 소양강댐	CQ1~CQ6 전체의 중심 대상
HydrometeorologicalState	댐과 연결되는 수문·기상 상태(강우, 수위, 유입량, 방류량 등) 변수 정보	충주댐_강우량	CQ1·CQ2
Operation	실제로 수행된 댐 운영행위	방류, 수문 개방	CQ3 (승인과 구분)
Approval	운영행위(방류/수문 운영 등)에 대한 승인	방류 승인	CQ4
Criterion	특정 Dam 에 적용되는 운영기준	홍수기 제한수위	CQ5
EvidenceSource	상태·운영·승인·기준을 확인할 수 있는 공식 근거자료	규정, 승인자료, 측정자료	CQ6

Relation (9 개)

Subject	Relation	Object	의미
Dam	hasHydrometeorologicalState	HydrometeorologicalState	댐이 해당 수문·기상 변수 정보와 연결됨
Operation	performedOnDam	Dam	운영행위가 특정 Dam 에서 수행됨
Approval	authorizes	Operation	Approval 이 특정 Operation 을 승인함
Approval	concernsDam	Dam	Approval 이 특정 Dam 에 관한 것임 (KG 구축 중 CQ4 에 답하기 위해 추가)
Criterion	appliesToDam	Dam	운영기준이 특정 Dam 에 적용됨
HydrometeorologicalState	supportedBy	EvidenceSource	상태를 확인할 수 있는 근거자료
Operation	supportedBy	EvidenceSource	운영행위를 확인할 수 있는 근거자료
Approval	supportedBy	EvidenceSource	승인을 확인할 수 있는 근거자료
Criterion	supportedBy	EvidenceSource	운영기준을 확인할 수 있는 근거자료

Data Property

Class	Data Property	필요한 이유
Dam	damName, damType	대상 댐을 식별하고 유형을 설명하기 위해
HydrometeorologicalState	variableType, stationCode	수문·기상 변수를 구분하고 측정자료를 찾아가기 위해
Operation	operationType, operationTime	어떤 운영이 언제 수행되었는지 확인하기 위해
Approval	approvalTime, approvalContent	언제 어떤 승인이 있었는지 확인하기 위해
Criterion	criterionType, criterionValue, unit	어떤 기준이 얼마의 값으로 적용되는지 확인하기 위해
EvidenceSource	sourceTitle, sourceType, sourceLocator	근거자료와 자료 내 근거 위치를 확인하기 위해

부록 D. KG 와 외부 저장소, 구축 대상 자료

구분	저장하는 것	예
KG	의미와 관계, 근거 위치, 외부 자료를 찾아가는 정보	승인·운영행위 Entity, 변수 종류와 측정소 코드, 근거자료의 위치
측정자료 저장소	시간별 실제 측정 시계열 값 전체	수위, 유입량, 방류량 등
문서 저장소	규정 원문	연계운영규정 별표 3 (현행)

KG 의 역할	KG 는 모든 데이터를 직접 저장하지 않고, 댐 운영에 필요한 상태·운영·승인·기준·근거의 의미 관계를 표현하며, 상세 문서와 대량 측정자료를 찾아갈 수 있도록 연결하는 중심 구조이다.

구축 대상 자료

자료	출처	범위	사용
방류 승인 기록	한강홍수통제소 댐방류승인(공공데이터포털)	충주·소양강·횡성·광동 4 개 댐 80 건 (2010-09 ~ 2021-07)	Approval, Operation
수문·기상 시간자료	K-water MyWater	4 개 댐 × 6 개 변수, 508,246 개 (댐, 시각). 충주·횡성 2010-08~, 소양강 2011-06~, 광동 2016-06~, 모두 2026-09 까지	측정자료 저장소, HMS
댐 일자료	K-water MyWater	4 개 댐의 댐 코드, 댐 이름, 댐 유형	Dam
홍수기 제한수위	연계운영규정 별표 3(법제처 XML)	충주 138.0, 소양강 190.3, 횡성 178.2 EL.m (광동댐은 별표 3 에 없음)	Criterion, 문서 저장소

부록 E. 근거 문헌의 원리

E.1 DDKG (Huang 외, 2026) — 단계 1, 단계 4

- 무엇인가: 중국의 결함·위험 댐 22,076 개의 안전 감정 보고서와 저수지 운영관리 플랫폼 자료로 댐 결함 지식그래프(DDKG, Dam Defects Knowledge Graph)를 만든 논문이다. 온톨로지 DDO(Dam Defects Ontology)의 최상위 Class 는 7 개다: c1 Dam, c2 Properties, c3 Location, c4 Parameter, c5 HazardForm(위험 형태), c6 HazardComponent(위험 부위), c7 SafetyEvaluation(안전평가).
- 3.2.1 지식 출처와 전처리: 플랫폼의 정형 표가 c1~c4 의 데이터를 대부분 채우고, 보고서의 비정형 텍스트(PDF, DOC, PNG)가 c5~c7 을 채운다. 비정형 텍스트는 마크다운으로 바꾸고(이미지는 OCR), 정규식으로 필요한 텍스트 구간의 경계를 찾아 잘라낸 뒤 하나의 말뭉치로 합친다. → 우리는 출처 구분(단계 1)만 따랐고, 전처리 기술은 쓰지 않았다.
- 3.2.2 규칙 기반 지식 추출: c7 안전평가 결과(홍수 방어, 침투, 구조, 내진, 금속 구조 각각 A/B/C 등급)는 고도로 표준화된 문형으로 쓰이므로 정규식으로 바로 뽑는다. 예: "홍수 방어 안전 등급은 C 등급이다" → C. → 단계 4 에서 따랐다.
- 3.2.3 LLM 기반 지식 추출: 표현이 제각각인 위험 서술(c5, c6)은 미세조정한 LLM, 온도를 바꾼 여러 번의 추론과 투표, 온톨로지 제약 검사, 전문가 검토로 분류한다(OHCF). → 우리 자료에는 이런 서술이 없어 쓰지 않았다.
- 3.3 저장: 추출한 지식을 JSON 으로 모은 뒤 Python 스크립트로 Cypher 문을 만들어 Neo4j 에 넣는다. → 단계 7(다음 주)에서 다룬다.

E.2 OntoDSMS 와 D2RQ (Zhou 외, 2023) — 단계 3

- 표기: Zhou 외(2023)는 댐 안전 모니터링을 위한 BIM·온톨로지 기반 지식관리 논문이다. OntoDSMS 는 이 논문이 만든 댐 안전 모니터링 시스템(DSMS) 도메인 온톨로지의 이름이다. D2RQ 는 이 논문이 만든 것이 아니라 기존 오픈소스 플랫폼이며, 논문은 MySQL 의 모니터링 데이터를 OntoDSMS 에 연결할 때 D2RQ 를 사용했다(4 장, 그림 3).
- D2RQ 는 "표는 Class, 열은 Data Property, 행은 Entity, 셀은 값"이라는 대응을 매핑 파일(Turtle 문법)의 선언으로 구현한다.
- 원리 1 (uriPattern): 식별자의 틀을 "item/@@item.item_ID@@"처럼 정하고, @@ 사이 열의 값을 행마다 끼워 넣는다. item_ID 가 7 인 행 → item/7. 같은 행은 언제나 같은 식별자를 받는다.
- 원리 2 (d2rq:class): "d2rq:class :Item"으로 그 표의 모든 행 식별자를 Item Class 의 Entity 로 선언한다. 논문: "Converting a table in a database to a class in an ontology"
- 원리 3 (PropertyBridge): 열마다 어느 Entity 에 붙일지(belongsToClassMap), 어떤 속성인지(property), 어느 열의 값을 옮길지(column)를 선언한다. 예: column "item.Genre" → 속성 :itemName. 고정 문자열과 열 값을 조합할 때는 pattern 을 쓴다. 논문: "Converting a column in a table to a DatatypeProperty in an ontology"
- 원리 4: 매핑 파일은 DB 구조로부터 자동 생성되며 이름은 DB 의 표·열 이름 그대로이다. 이를 사람이 온톨로지 용어에 맞게 고친다(Genre 열 → :itemName).

E.3 GraphAide (Purohit 외, 2024) — 단계 5

- 무엇인가: LLM, 벡터 DB, 그래프 DB 를 함께 써서 온톨로지에 맞는 KG 를 만들고, 자연어 질문에 답하는 시스템의 구조를 제시한 논문이다. KG 생성 단계는 개체명 추출, 관계 추출, 개체 중의성 해소(entity disambiguation), KG 인코딩, KG 유지보수로 이루어진다.
- 개체 중의성 해소: 실험에서 DARPA 판 Wikidata(DWD)를 기준 지식베이스로 두고, 각 고유번호(QID)의 설명을 벡터 DB 에 넣었다. 뉴스 기사 1,846 건에서 뽑은 이름(mention)마다 벡터 검색으로 후보 QID 를 찾고, LLM 이 후보의 순위를 다시 매겨 가장 가까운 QID 에 대응시켰다.

- 원리: 자료마다 다르게 적힌 대상을 기준 목록의 고유 식별자 하나에 대응시키고, 그 식별자로 잇는다. → 단계 5 에서 기준 식별자를 K-water 댐 코드(DAM_CD)로 두고 따랐다. 우리 자료에는 코드나 정해진 이름이 있어 벡터 검색과 LLM 없이 기준 목록에서 바로 찾는다.

부록 F. 참고문헌

- Huang, J., Hou, W., Zhang, S., Du, Z., & Yin, J. (2026). Dam defects ontology and knowledge graph construction from multi-source hazard data using large language models. Advanced Engineering Informatics, 76, 104987.
- Zhou, Y., Bao, T., Shu, X., Li, Y., & Li, Y. (2023). BIM and ontology-based knowledge management for dam safety monitoring. Automation in Construction, 145, 104649.
- Purohit, S., Chin, G., Mackey, P. S., & Cottam, J. A. (2024). GraphAide: Advanced Graph-Assisted Query and Reasoning System. IEEE BigData.